Sunny Hollow AI Front Desk — Exercise Submission

Zane Meyer · Staff AI Software Engineer, Product · brightwheel
Parent chat: https://ai-front-desk-eight-woad.vercel.app · **Operator control center:** https://ai-front-desk-eight-woad.vercel.app/admin
Code: https://github.com/zanemeyer/ai-front-desk · **Focus:** Breadth, with a trust spine · **Tools:** Cursor, Gemini 2.5 Flash (free tier), Next.js, Vercel, Neon · **Time:** ~3 h core build + a second pass (below). Fictional center, invented policies, no real data.

Problem understanding

Admins answer the same questions all day while anxious parents wait on voicemail. An AI Front Desk earns trust only if answers are **center-specific, citable, and honest about uncertainty**, and earns adoption only if a busy operator can fix a wrong or missing answer in under a minute.

What I built, against the brief

Parent experience (front desk) — mobile web chat plus a Twilio SMS webhook.

- *Ask a question:* free text with suggested starters, multi-turn so "what about toddlers?" works after a tuition question.
- *Center-specific, trustworthy:* every answer cites handbook entries with tap-to-expand source quotes and a confidence level; a freshness caveat appears when a weekly-changing entry (lunch menu) is overdue.
- *Uncertain or sensitive:* ambiguous questions get **one clarifying question with tappable chips**; unknowns fail closed ("I don't want to guess; I've sent this to the office"); sensitive situations (medication, incidents, disputes, custody, staff complaints) are caught by hard rules plus a classifier, **staff are pinged automatically**, and the parent is told who.

Operator experience (control center)

- *Provide or edit the source of truth:* handbook editor with draft/publish, effective-date windows, and review cadence; the sensitive-situation list is a setting, not code.
- *What's asked and where it struggled:* hours saved, auto-answer rate, top topics, and a **Needs-attention inbox clustered by intent** (three sibling-discount questions = one card, ×3) showing *why* each failed: model reason, rejected citation IDs, confidence, thumbs-down text.
- *Improve over time:* **Answer → AI drafts a handbook entry → edit → publish → re-test**, resolving every question in the cluster; edits apply to the next question.

The five questions from the brief: Veterans Day → "No — we are closed…" with the closures entry cited; infant tuition → `$385/week` cited; fever → the 24-hour fever-free policy, answered rather than escalated (the classifier distinguishes illness *policy* from medication *requests*); forgot lunch → yes, first emergency meal free, plus today's menu resolved from the date; tour → how to schedule, with hours.

Beyond the brief, in a second pass: (1) Postgres persistence with versioned seeds; (2) "why it struggled" detail per question; (3) clarifying questions with chips; (4) hours-saved metric; (5) dedicated sensitivity classifier with measured precision/recall, auto-pinging staff and telling the parent; (6) entry freshness: effective-date windows, review cadence, stale caveats; (7) evals in CI with a 10-minute budget and a `promptVersion` on every answer; (8) multi-turn context; (9) inbox clustered by intent with counts; (10) parent "this looks outdated" flags that target one entry; (11) a Twilio SMS channel on the same pipeline.

Three decisions worth defending

1. **Whole handbook in context, not vectors.** At ~15 policies, retrieval adds a failure mode without value. Trust comes from a **quote-then-answer** schema: the model copies the supporting sentence verbatim *before* writing the answer, and the server verifies the quote and citation IDs against the handbook (unverifiable → confidence downgraded, shown to staff; unknown ID → rejected). Swap in retrieval at a few hundred entries.
2. **Fail closed, but ask before failing.** A confident wrong answer is worse than a warm escalation; a clarifying question beats both. Yes/no polarity is checked server-side too: the first build said "Yes, we are closed on Veterans Day."
3. **The inbox authors the handbook.** Real parent demand, clustered by intent, is the content backlog. That is what makes it *out-of-the-box*: a center starts thin and fills in from the questions parents actually ask.

How I know it works

`npm run eval`: 26 golden questions asserting outcome (answered / clarify / gap / escalation), key facts, polarity, multi-turn, and false-positive guards ("Is the registration fee refundable?" must be answered, not escalated on "refund"): **26/26 on production**. `npm run eval:sensitive`: 30 labeled messages, **100% precision / 100% recall** on the needs-a-human gate (v1 was 88% precision; the misses were the fever and refund questions, fixed by rewording topic descriptions). Both run in CI on every push.

What I'd do next with more time

Staff login on `/admin` and Twilio signature validation; staff pings via brightwheel messaging/push; off-topic detection so "who is the muffin man" gets a friendly redirect, not an inbox card; embedding-based intent dedup; multi-tenant handbooks with guided import; retrieval once the handbook outgrows the context window.